

3.4 GPT and BERT

Same block. Causal next-token versus masked tokens. Contextual, not a static table.

These notes match the lecture slides. Use **(slides)** on the course hub for the deck.

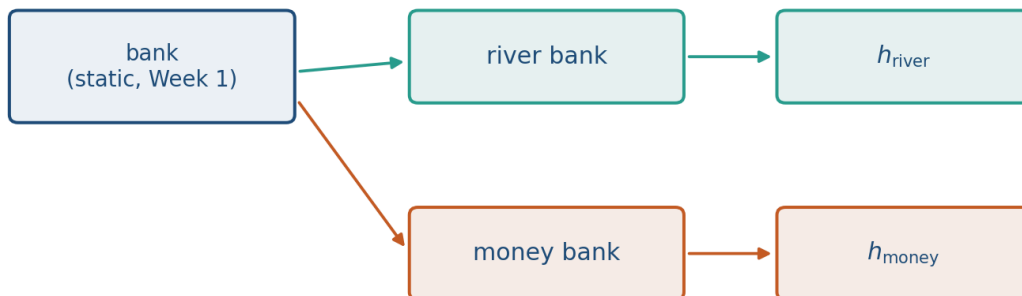
GPT and BERT are the same block with different **masks** and **objectives**. That choice is the difference between a generator and a bidirectional encoder. The fork is the objective: causal language modeling versus masked language modeling. By the end you should be able to fill the objective table, mask a 2×2 score matrix, and say why BERT cannot stream tokens.

1. What GPT and BERT are

GPT and **BERT** are two uses of the same transformer block. GPT: causal next-token (left to right). BERT: masked tokens, bidirectional context. Both produce **contextual** vectors, not a static table (Week 1).

Week 1's embedding table gave *bank* **one** vector. A transformer gives *bank* a vector that depends on the sentence: river bank versus money bank. That is a **contextual** representation. ELMo did this with LSTMs; GPT and BERT do it with the Week 3 block.

A transformer embedding is contextual: the same type, two vectors



"Transformer" is the block. GPT/BERT is the **objective**. Calling every transformer "a GPT" erases that fork.

2. Why we use it

Language modeling (GPT) is the pretrain that later becomes ChatGPT. BERT-style MLM is a bidirectional encoder for classification and span tasks.

Pretrain once on unlabeled text, then fine-tune a small head (or the whole stack) on a labeled task.

Pretrain once, fine-tune many times.

The same block, bigger: more layers, wider d , more heads, more data. Week 7 is scaling laws and efficiency. This week: if the mask is causal, you have a language model; if not, you have a contextual encoder.

For the project: generation, RAG, and tool-use sit on a decoder. Classification into a small label set can still use an encoder, or a decoder with a prompt.

Do not fine-tune BERT as if it were GPT. A [MASK] model is not a chat decoder. You *can* classify with a decoder via a prompt; you cannot stream tokens from BERT without a separate head.

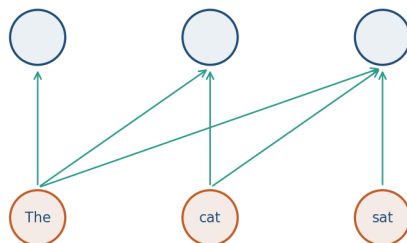
3. Architecture

Same transformer block as note 3.3. GPT masks the future. BERT masks random token ids and reconstructs them. Heads differ (LM versus CLS/span).

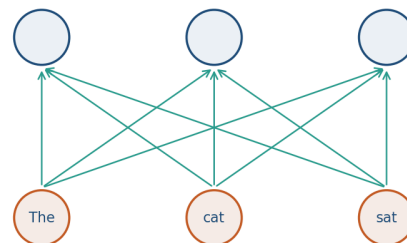
GPT (decoder-only): each token may attend to itself and the past. The pretraining job is **next-token** (causal language modeling). Sampling is ancestral: emit t , then condition on it.

BERT (encoder-only): every token may attend to every token. The pretraining job is **masked language modeling** (and originally next-sentence prediction; later work often drops NSP). You cannot sample a paragraph left-to-right without extra machinery.

GPT: causal (next token)

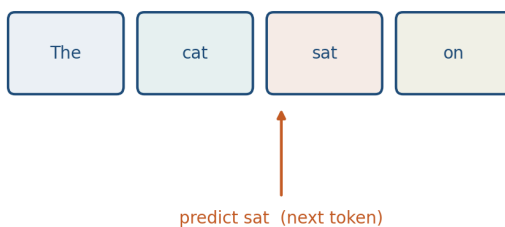


BERT: bidirectional (masked token)

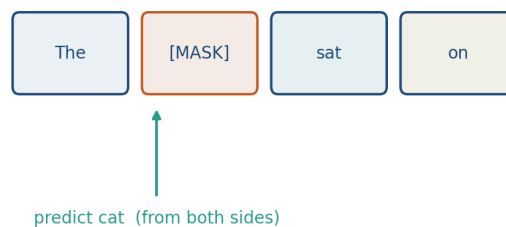


Causal mask: a lower-triangular pattern of legal scores. BERT's "mask" is a **token** [MASK] in the input, not a future-key mask. Those two uses of the word confuse people all semester.

GPT: causal language model



BERT: masked language model



	GPT	BERT
Stack	decoder	encoder
Mask	causal	none on keys (random tokens hidden)
Loss	$p(x_t \mid x_{1:t-1})$	$p(x_{\text{masked}} \mid x_{\text{rest}})$
Default use	generate, chat, complete	classify, tag, embed a span
Peek at the future?	no	yes, by design

Modern “BERT-like” encoders (RoBERTa, embedding models) keep the bidirectional idea. Modern LLMs are GPT-style stacks, often with extra alignment (Weeks 8–9). Encoder–decoder models (T5, BART) exist; they are the original transformer, not a third mystery architecture.

BERT-base, one line: $N = 12$, $d = 768$, 12 heads, about 110M parameters. GPT-1 was the same size on a decoder. The fork is the mask and the loss, not the width.

4. How it works, step by step

Train on unlabeled text. After pretrain, GPT generates; BERT is usually fine-tuned with a small head.

GPT path.

1. Embed tokens and add positions. Apply a causal mask so token t cannot see $t + 1, \dots, T$.
2. The loss is next-token NLL: $-\log p(x_t \mid x_{<t})$ at each position (teacher forcing, as in the RNN-LM).
3. At generation time, sample $\hat{x}_t$, append it, and continue. That is ancestral sampling.

BERT path.

1. Hide a random subset of token ids (classically about 15%) by replacing them with [MASK] (and some random / keep-id noise in the original recipe).
2. Every token may attend to every token. There is no future-key mask.
3. The loss reconstructs the **masked** positions, $-\log p(x_{\text{masked}} \mid x_{\text{rest}})$, not a full joint factorization of the sentence.
4. Downstream, add a small head on [CLS] or on token states and fine-tune.

Masking GPT scores with 0 instead of $-\infty$ is the same bug as Lab 3: softmax still puts mass on the illegal key.

5. Mathematical formulas

GPT factorizes the sentence left to right:

$$p(x) = \prod_t p(w_t \mid w_{<t}).$$

BERT reconstructs masked positions, not a full joint factorization:

$$p(x_{\text{masked}} \mid x_{\text{rest}}).$$

A causal mask sets illegal future scores to $-\infty$ before the softmax, so those probabilities are 0. BERT has no such triangular mask; its [MASK] is an input token.

Worked 2×2 scores live in the example below; the algebra is row-wise softmax on the (possibly masked) score matrix.

6. Positive points and negative points

Positive.

- Contextual geometry: *bank* in “river bank” is not the same vector as *bank* in “money bank.”
- One block, two objectives: you reuse note 3.3 and only change the mask and the loss.
- GPT is the default stack for generation, RAG, and tool-use in this course.
- BERT-style encoders remain a strong default for classify / tag / embed-a-span when you do not need to stream tokens.
- Pretrain once, fine-tune many times, on unlabeled text then a small labeled head.

Negative.

- GPT cannot see the future inside a prompt token; a bidirectional encoder can, by design.
- BERT is not a natural generator. You cannot stream tokens from it without a separate head.
- [MASK] is not the same object as a causal key mask; mixing those words lasts all semester.
- Calling every transformer “a GPT” hides the objective fork.
- Masking GPT scores with 0 instead of $-\infty$ lets probability leak to the future.

When not to. If the demo must **stream** tokens, BERT is the wrong pretrained model. If you only need a small label set and full-sentence context, a decoder is optional, not mandatory.

7. Teaching this note

~**16 minutes.** Static vs contextual *bank*, then the two attention triangles, then the objective table, then the 2×2 causal-softmax example. Play Karpathy **Let’s build GPT 0:00–12:00** (tokens in, next token out). Then **0:00–8:00** of CodeEmporium BERT (MLM / [MASK]). If you only have one clip, keep Karpathy and assign BERT.

8. Worked example

Two tokens, raw scores $S = \begin{bmatrix} 1 & 2 \\ 3 & 4 \end{bmatrix}$.

GPT (causal): illegal future is S_{12} . Set it to $-\infty$:

$$S_{\text{GPT}} = \begin{bmatrix} 1 & -\infty \\ 3 & 4 \end{bmatrix}.$$

Softmax row 1: $[1, 0]$. Row 2: $\text{softmax}([3, 4]) \approx [0.27, 0.73]$ because $e^3 \approx 20.1$, $e^4 \approx 54.6$.

BERT (bidirectional): softmax both rows of the original S . Row 1: $\text{softmax}([1, 2]) \approx [0.27, 0.73]$. Token 1 **does** look at token 2.

Objective contrast: GPT loss on this pair is $-\log p(x_2 | x_1)$. BERT, if token 1 is [MASK], is $-\log p(x_1 | x_2)$ (and any other masked sites).

9. Where students get stuck

- Using BERT when the demo must **stream** tokens.
 - Masking GPT scores with 0 instead of $-\infty$.
 - Calling every transformer “a GPT.”
 - Thinking [MASK] is the same object as a causal key mask.
-

10. Video

Watch [Karpathy: Let's build GPT](#) through the first attention / causal-mask implementation (start **0:00–12:00** in class; the attention code is later if you assign homework).

Also [CodeEmporium: BERT Neural Network — EXPLAINED!](#). Pause on masked language modeling and next-sentence prediction. You do not need to train BERT today.

11. Practice

1. Why is BERT the wrong pretrained model if your demo must stream tokens?
2. A sentiment classifier: encoder or decoder? Give one reason for each.
3. What does a causal mask set the illegal scores to before the softmax, and why not zero?
4. Scores $\begin{bmatrix} 0 & 0 \\ 0 & 0 \end{bmatrix}$, causal mask. Write A . (Uniform among **legal** keys.)
5. BERT hides 15% of tokens. In a sequence of length 20, how many tokens are masked in expectation?
If the loss is only on those positions, how many terms enter the mean that step?

6. *bank* in “river bank” versus “money bank”: what did Week 1’s table do, and what does BERT do?
7. Name one encoder-only, one decoder-only, and one encoder–decoder model. Which one is the default stack for this course’s generation projects?